

LABORATORY OBSERVATION

Course	Full Stack Web Development
Course Code	23CM4121
Experiment No.	4
Student Name	K. Anusha
Roll No.	A24126552088
Date	29-08-2026

1. TITLE

To-Do List Web Application – DOM Manipulation and Event Handling using HTML, CSS, and JavaScript

2. AIM

To design and develop a simple To-Do List web application using HTML, CSS, and JavaScript, and to understand the concepts of DOM element selection, dynamic element creation, event handling, and input validation by executing the application and observing its behavior for different test cases.

3. OBJECTIVE

The objectives of this experiment are:

- To create the structure of a web page using HTML elements such as div, h1, input, button, ul, and p.
- To style the web page using an external CSS file with flexbox layout, class selectors, and hover effects.
- To select HTML elements in JavaScript using `document.getElementById()`.
- To create new HTML elements dynamically using `document.createElement()` and add them to the page using `appendChild()`.
- To handle user actions using `addEventListener()` for click and keypress events.
- To validate user input using `trim()` and display a warning using `alert()` when the input is empty.
- To mark a task as completed or not completed using `classList.toggle()`.
- To delete a task from the page using the `remove()` method.
- To show or hide the empty-list message using the `style.display` property.

4. THEORY

A web application is built using three core technologies: HTML for structure, CSS for presentation, and JavaScript for behavior. In this experiment, all three are used together to build an interactive To-Do List where the user can add, complete, and delete tasks without reloading the page.

HTML – Structure

HTML (HyperText Markup Language) defines the structure of a web page using elements. In this experiment, a div with class container holds the heading, an input box, the Add Task button, an empty unordered list (ul) that will store the tasks, and a paragraph that displays a message when there are no tasks. Elements are given id and class attributes so that CSS and JavaScript can identify them.

CSS – Styling

CSS (Cascading Style Sheets) controls the appearance of the page. An external stylesheet is linked to the HTML file using the link tag. Flexbox (display: flex) arranges items in a row, gap adds spacing between them, and justify-content: space-between places the task text on the left and the buttons on the right. Class selectors such as .task-item and .completed style groups of elements, the :hover pseudo-class changes the look of a button when the mouse pointer is over it, and text-decoration: line-through draws a line across the text of a completed task.

DOM – Document Object Model

When a browser loads an HTML page, it represents the page as a tree of objects called the DOM. JavaScript can use the DOM to find, create, change, and remove elements while the page is running. The main DOM operations used in this experiment are:

- `document.getElementById()` – selects an element using its id.
- `document.createElement()` – creates a new element in memory.
- `textContent` and `className` – set the text and CSS class of an element.
- `appendChild()` – inserts an element inside another element.
- `remove()` – deletes an element from the page.
- `classList.toggle()` – adds a class if it is absent and removes it if it is present.
- `style.display` – shows or hides an element.

Events and Event Handling

An event is an action performed by the user, such as clicking a button or pressing a key. The `addEventListener()` method attaches a function to an element so that the function runs when the specified event occurs. In this experiment, a click listener is attached to the Add Task, Complete, and Delete buttons, and a keypress listener is attached to the input box. The `event.key` property is used to detect whether the Enter key was pressed, and calling `addTaskBtn.click()` reuses the same add-task logic without repeating code.

Input Validation

User input should be checked before it is used. The `trim()` method removes extra spaces from the beginning and end of the text. If the result is an empty string, the program displays a warning using `alert()` and stops using `return`, so that blank tasks are never added to the list.

The basic concepts demonstrated in this experiment are:

HTML Structure → CSS Styling → DOM Selection → Event Handling → Dynamic Element Creation → Toggle / Remove → UI Update

5. ALGORITHM / PROCEDURE

1. Create a project folder for the To-Do List application.
2. Create three files inside the folder: index.html, style.css, and script.js.
3. In index.html, create a container div containing a heading, an input box (taskInput), an Add Task button (addTaskBtn), an empty list (taskList), and a paragraph (emptyMessage).
4. Link style.css in the head section and script.js at the end of the body section of index.html.
5. In style.css, style the body, container, input section (using flexbox), input box, buttons, task items, completed tasks, and the empty message.
6. In script.js, select the required HTML elements using document.getElementById().
7. Create the function updateEmptyMessage() to display the message when the task list has no children and hide it otherwise.
8. Add a click event listener to the Add Task button.
9. Read the text from the input box using value.trim(). If it is empty, display an alert message and stop.
10. Otherwise, create a list item (li) with class task-item and a span with class task-text that holds the entered text.
11. Create a Complete button and add a click listener that toggles the completed class on the task text.
12. Create a Delete button and add a click listener that removes the task item and calls updateEmptyMessage().
13. Append the text, Complete button, and Delete button to the list item, and append the list item to the task list.
14. Clear the input box and call updateEmptyMessage().
15. Add a keypress event listener to the input box so that pressing Enter triggers the Add Task button.
16. Call updateEmptyMessage() once when the page loads.
17. Open index.html in a web browser and execute all the test cases.
18. Observe and record the output of each test case.

6. PROGRAM

Program 1: index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
  <title>My To-Do List</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>My To-Do List</h1>
    <div class="input-section">
      <input type="text" id="taskInput" placeholder="Enter a task">
      <button id="addTaskBtn">Add Task</button>
    </div>
    <ul id="taskList"></ul>
    <p id="emptyMessage">
      No tasks available.
    </p>
  </div>
  <script src="script.js"></script>
</body>
</html>
```

Program 2: style.css

```

body {
  font-family: Arial, sans-serif;
  background-color: #f2f4f7;
  margin: 0;
  padding: 40px;
}
.container {
  width: 600px;
  max-width: 90%;
  margin: auto;
  padding: 25px;
  background-color: rgb(242, 237, 237);
  border-radius: 10px;
  box-shadow: 0 4px 12px rgb(27, 109, 153);
}
h1 {
  text-align: center;
  color: #000000;
}
.input-section {
  display: flex;
  gap: 10px;
}
#taskInput {
  flex: 1;
  padding: 12px;
  border: 1px solid #46134b;
  border-radius: 5px;
  font-size: 15px;
}

```

```

#addTaskBtn {
  padding: 12px 18px;
  background-color: #8f1737;
  color: rgb(240, 232, 232);
  border: none;
  border-radius: 5px;
  cursor: pointer;
}
#addTaskBtn:hover {
  background-color: #22705f;
}
#taskList {
  list-style: none;
  padding: 0;
  margin-top: 25px;
}
.task-item {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 12px;
  margin-bottom: 10px;
  background-color: #ac97c0;
  border: 1px solid #049b2f;
  border-radius: 5px;
}
.task-text {
  flex: 1;
  font-size: 16px;
}

```

```

}
.completed {
    text-decoration: line-through;
    color: rgb(0, 0, 0);
}
.complete-btn {
    margin-left: 10px;
    padding: 8px 12px;
    background-color: #08530d;
    color: white;
    border: none;
    border-radius: 4px;
    cursor: pointer;
}
.delete-btn {
    margin-left: 8px;
    padding: 8px 12px;
    background-color: #c71010fb;
    color: white;
    border: none;
    border-radius: 4px;
    cursor: pointer;
}
.complete-btn:hover {
    background-color: #e20707;
}
.delete-btn:hover {
    background-color: #a82c2c;
}

```

```

    cursor: pointer;
}
.complete-btn:hover {
    background-color: #e20707;
}
.delete-btn:hover {
    background-color: #a82c2c;
}
#emptyMessage {
    text-align: center;
    color: #0d5444;
    font-style: italic;
}

```

Program 3: script.js

```

// Function to update the empty-list message
function updateEmptyMessage() {
  // Check whether the task list contains any tasks
  if (taskList.children.length === 0) {
    emptyMessage.style.display = "block";
  } else {
    emptyMessage.style.display = "none";
  }
}

// Add a new task when the Add Task button is clicked
addTaskBtn.addEventListener("click", function () {
  // Get the task entered by the user
  const taskText = taskInput.value.trim();
  // Check whether the input is empty
  if (taskText === "") {
    alert("Please enter a task.");
    return;
  }
  // Create a new list item
  const taskItem = document.createElement("li");
  // Add CSS class to the list item
  taskItem.className = "task-item";
  // Create element for task text
  const task = document.createElement("span");
  task.className = "task-text";
  task.textContent = taskText;
  // Create Complete button
  const completeBtn = document.createElement("button");
  completeBtn.className = "complete-btn";

```

```

  completeBtn.textContent = "Complete";
  // Add click event to Complete button
  completeBtn.addEventListener("click", function () {
    // Add or remove the completed class
    task.classList.toggle("completed");
  });
  // Create Delete button
  const deleteBtn = document.createElement("button");
  deleteBtn.className = "delete-btn";
  deleteBtn.textContent = "Delete";
  // Add click event to Delete button
  deleteBtn.addEventListener("click", function () {
    // Remove the selected task from the webpage
    taskItem.remove();
    // Update empty-list message
    updateEmptyMessage();
  });
  // Add elements to the task item
  taskItem.appendChild(task);
  taskItem.appendChild(completeBtn);
  taskItem.appendChild(deleteBtn);
  // Add the task item to the task list
  taskList.appendChild(taskItem);
  // Clear the input box
  taskInput.value = "";
  // Update empty-list message
  updateEmptyMessage();
}

```

```

});
// Allow the Enter key to add a task
taskInput.addEventListener("keypress", function (event) {

    if (event.key === "Enter") {
        addTaskBtn.click();
    }

});
// Display the empty message when the page first loads
updateEmptyMessage();

```

7. CODE EXPLANATION

Code / Concept	Explanation
<div class="container">	Main wrapper that holds the heading, input section, task list, and empty message.
<input id="taskInput">	Text box in which the user types a new task.
<button id="addTaskBtn">	Button used to add the entered task to the list.
<ul id="taskList">	Empty unordered list; tasks are added to it dynamically by JavaScript.
<p id="emptyMessage">	Displays "No tasks available." when the list is empty.
<link rel="stylesheet">	Connects the external CSS file to the HTML page.
<script src="script.js">	Loads the JavaScript file at the end of the body so that all elements exist before the script runs.
display: flex; gap: 10px;	Places the input box and button side by side with spacing between them.
.task-item	Styles each task row; flex with space-between keeps the text on the left and buttons on the right.
.completed	Draws a line through the text of a completed task using textDecoration: line-through.
:hover	Changes the background color of a button when the mouse pointer is over it.
document.getElementById()	Selects an HTML element using its id.
function updateEmptyMessage()	Shows the empty message if taskList.children.length is 0, otherwise hides it.

<code>addEventListener("click", ...)</code>	Runs the given function when the button is clicked.
<code>taskInput.value.trim()</code>	Reads the entered text after removing leading and trailing spaces.
Code / Concept	Explanation
<code>alert("Please enter a task.")</code>	Warns the user when the input is empty; return then stops the function.
<code>document.createElement("li")</code>	Creates a new list item element for the task.
<code>className, textContent</code>	Set the CSS class and the text of a newly created element.
<code>classList.toggle("completed")</code>	Adds the completed class if it is absent and removes it if it is present.
<code>taskItem.remove()</code>	Removes the selected task from the web page.
<code>appendChild()</code>	Adds the text and buttons to the list item, and the list item to the task list.
<code>taskInput.value = ""</code>	Clears the input box after a task is added.
<code>event.key === "Enter"</code>	Checks whether the Enter key was pressed inside the input box.
<code>addTaskBtn.click()</code>	Triggers the Add Task button programmatically, reusing the same addtask code.
<code>updateEmptyMessage();</code> (last line)	Sets the correct message state when the page first loads.

8. TEST CASES AND EXECUTION DATA

Group 1: Page Load, Adding Tasks, and Validation

Test Case	Operation	Expected Result	Actual Result	Status
TC-01	Open index.html in the browser	Heading, input box, Add Task button, and "No tasks available." message should be displayed	Page displayed with heading "My To-Do List", input placeholder "Enter a task", and message "No tasks available."	Pass
TC-02	Enter "Complete JavaScript lab record" and click Add Task	Task should be added with Complete and Delete buttons, the empty message hidden, and the input box cleared	Task added with Complete and Delete buttons; empty message hidden; input box cleared	Pass

TC-03	Click Add Task with the input box empty	Alert "Please enter a task." should be shown and no task added	Alert "Please enter a task." displayed; task count remained 1	Pass
TC-04	Enter only spaces and click Add Task	Input should be treated as empty; alert shown and no task added	Alert "Please enter a task." displayed; task count remained 1	Pass
TC-05	Add "Practice DOM methods" and "Submit Week 4 record"	Both tasks should be added below the existing task in the same order	3 tasks displayed in the order they were added	Pass
Test Case	Operation	Expected Result	Actual Result	Status
TC-06	Type "Revise event handling" and press Enter	Task should be added without clicking the button	Task added using the Enter key; input box cleared; 4 tasks displayed	Pass

Group 2: Completing Tasks

Test Case	Operation	Expected Result	Actual Result	Status
TC-07	Click Complete on the second task	Task text should get a linethrough; other tasks unchanged	Task text struck through (class "task-text completed"); other tasks unchanged	Pass
TC-08	Click Complete again on the same task	Line-through should be removed	Line-through removed (class "task-text")	Pass

Group 3: Deleting Tasks

Test Case	Operation	Expected Result	Actual Result	Status
TC-09	Click Delete on the first task	Task should be removed and the remaining tasks stay	Task removed; 3 tasks remain	Pass
TC-10	Delete all remaining tasks	List should be empty and "No tasks available." message should reappear	All tasks removed; message "No tasks available." displayed again	Pass

9. OUTPUT

The application (index.html) was opened in a web browser and each test case below was executed. The screenshot for each test case shows the state of the page after the operation. For TC-03 and TC-04, the browser displayed the alert message "Please enter a task."; the screenshots show the page after the alert was closed, with no new task added.

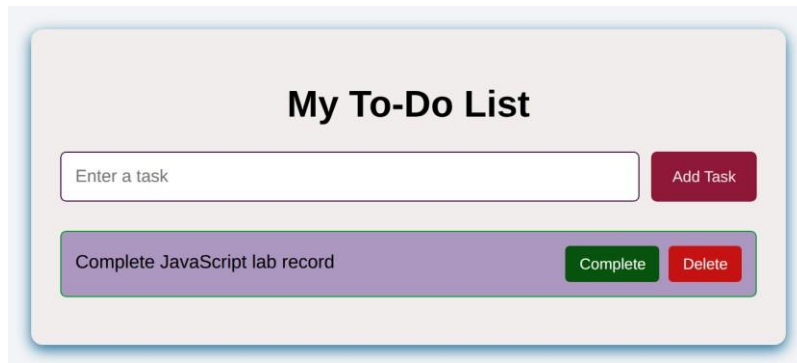
Output 1 (TC-01): Open index.html (empty list)

My To-Do List

Add Task

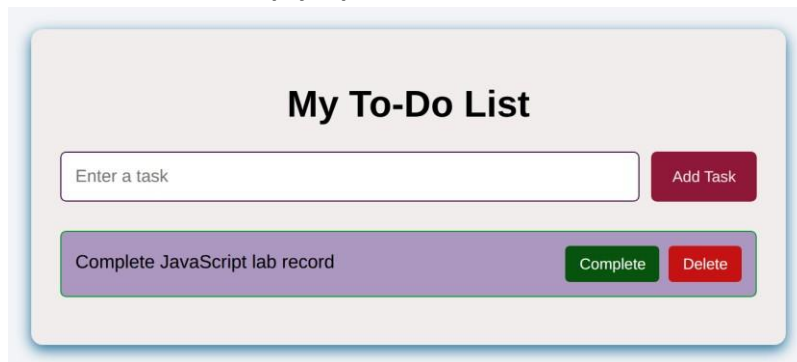
No tasks available.

Output 2 (TC 02): Add a valid task



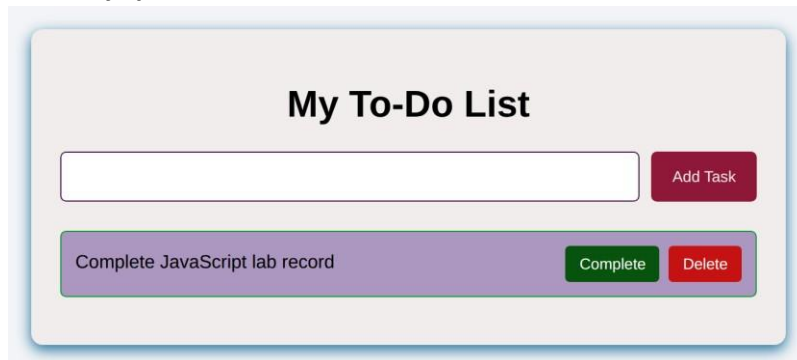
The screenshot shows a web application titled "My To-Do List". At the top, there is a text input field with the placeholder text "Enter a task" and a red "Add Task" button to its right. Below the input field, there is a purple task card that contains the text "Complete JavaScript lab record". To the right of the task card are two buttons: a green "Complete" button and a red "Delete" button.

Output 3 (TC-03): Click Add Task with empty input



This screenshot is identical to the previous one, showing the "My To-Do List" interface with the task "Complete JavaScript lab record" and the "Add Task" button.

Output 4 (TC-04): Enter only spaces and click Add Task



The screenshot shows the "My To-Do List" interface. The text input field is now empty, indicating that only spaces were entered and then cleared. The "Add Task" button remains to the right of the input field. The task card "Complete JavaScript lab record" with "Complete" and "Delete" buttons is still visible below.

Output 5 (TC 05): Add multiple tasks

My To-Do List

Add Task

Complete JavaScript lab record

CompleteDelete

Practice DOM methods

CompleteDelete

Submit Week 4 record

CompleteDelete

Output 6 (TC-06): Add a task using the Enter key

My To-Do List

Add Task

Complete JavaScript lab record

CompleteDelete

Practice DOM methods

CompleteDelete

Submit Week 4 record

CompleteDelete

Revise event handling

CompleteDelete

Output 7 (TC 07): Click Complete on the second task

My To-Do List

Add Task

Complete JavaScript lab record

CompleteDelete

Practice DOM methods

CompleteDelete

Submit Week 4 record

CompleteDelete

Revise event handling

CompleteDelete

Output 8 (TC-08): Click Complete again (undo)

My To-Do List

Add Task

Complete JavaScript lab record

CompleteDelete

Practice DOM methods

CompleteDelete

Submit Week 4 record

CompleteDelete

Revise event handling

CompleteDelete

Output 9 (TC 09): Delete the first task

My To-Do List

Enter a task Add Task

Practice DOM methods Complete Delete

Submit Week 4 record Complete Delete

Revise event handling Complete Delete

Output 10 (TC-10): Delete all tasks

My To-Do List

Enter a task Add Task

No tasks available.

10. RESULT

Thus, a To-Do List web application was successfully designed and implemented using HTML, CSS, and JavaScript. The page structure was created with HTML and styled with an external CSS file using flexbox and class-based styles. DOM methods such as `getElementById()`, `createElement()`, `appendChild()`, and `remove()` were used to select, create, and delete elements dynamically. Event handling was implemented using `addEventListener()` for the click and keypress events, input validation was performed using `trim()` and `alert()`, and tasks were marked as completed using `classList.toggle()`. All test cases were executed successfully and the expected outputs were obtained, with the output captured for each test case above.